encounter any "weird issues" with YAML dictionaries, try putting the relevant portion of your dictionary into a Ruby file.)

3.2 Passing variables to translations

You can use variables in the translation messages and pass their values from the view.

```
# app/views/home/index.html.erb
<%=t 'greet_username', user: "Bill", message: "Goodbye" %>
# config/locales/en.yml
en:
  greet_username: "%{message}, %{user}!"
```

3.3 Adding Date/Time Formats

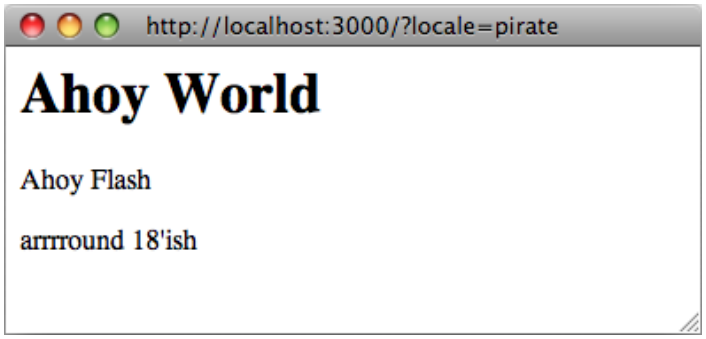
OK! Now let's add a timestamp to the view, so we can demo the **date/time localization** feature as well. To localize the time format you pass the Time object to `I18n.l` or (preferably) use Rails' `#l`helper. You can pick a format by passing the `:format` option - by default the `:default` format is used.

```
# app/views/home/index.html.erb
<h1><%=t :hello_world %></h1>
<p><%= flash[:notice] %></p
<p><%= l Time.now, format: :short %></p>
```

And in our pirate translations file let's add a time format (it's already there in Rails' defaults for English):

```
# config/locales/pirate.yml
pirate:
  time:
    formats:
      short: "arrrround %H'ish"
```

So that would give you:



Right now you might need to add some more date/time formats in order to make the I18n backend work as expected (at least for the 'pirate' locale). Of course, there's a great chance that somebody already did all the work by **translating Rails' defaults for your locale**. See the rails-i18n repository at GitHub for an archive of various locale files. When you put such file(s) in `config/locales/` directory, they will automatically be ready for use.

3.4 Inflection Rules For Other Locales

Rails allows you to define inflection rules (such as rules for singularization and pluralization) for locales other than English. In `config/initializers/inflections.rb`, you can define these rules for multiple locales. The initializer contains a default example for specifying additional rules for English; follow that format for other locales as you see fit.